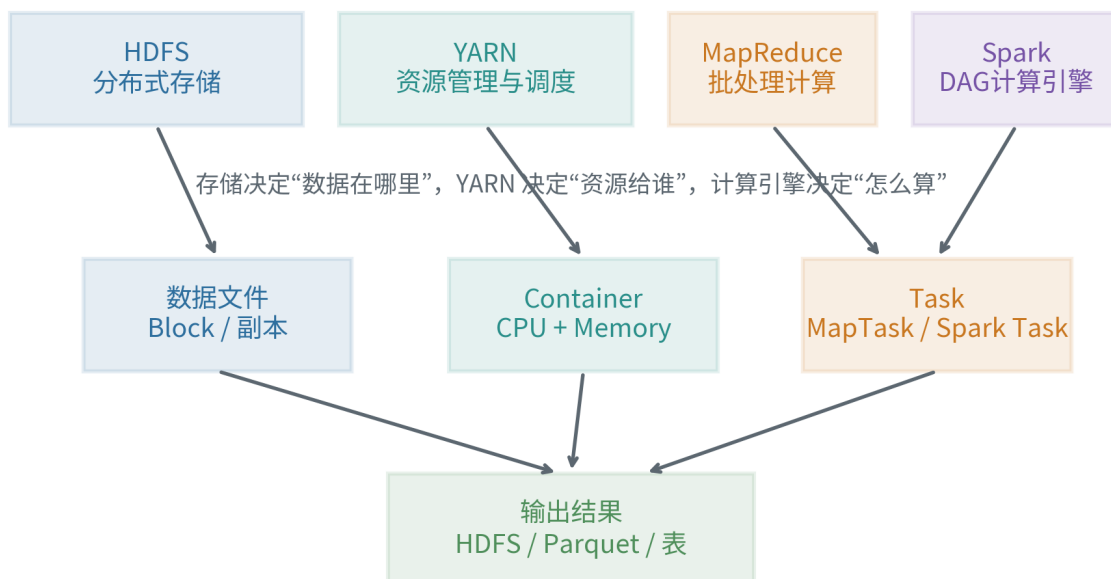


第20讲

Hadoop + MapReduce + Spark 综合收束

把 HDFS、YARN、MapReduce、Spark、RDD、DataFrame、Catalyst、Shuffle、DAG、Job、Stage、Task 放进同一张知识地图

Hadoop + Spark 统一知识地图



本讲目标：不再增加新 API，而是建立“能解释、能排错、能迁移”的完整心智模型。

课程阶段：MapReduce + Spark 最终闭环

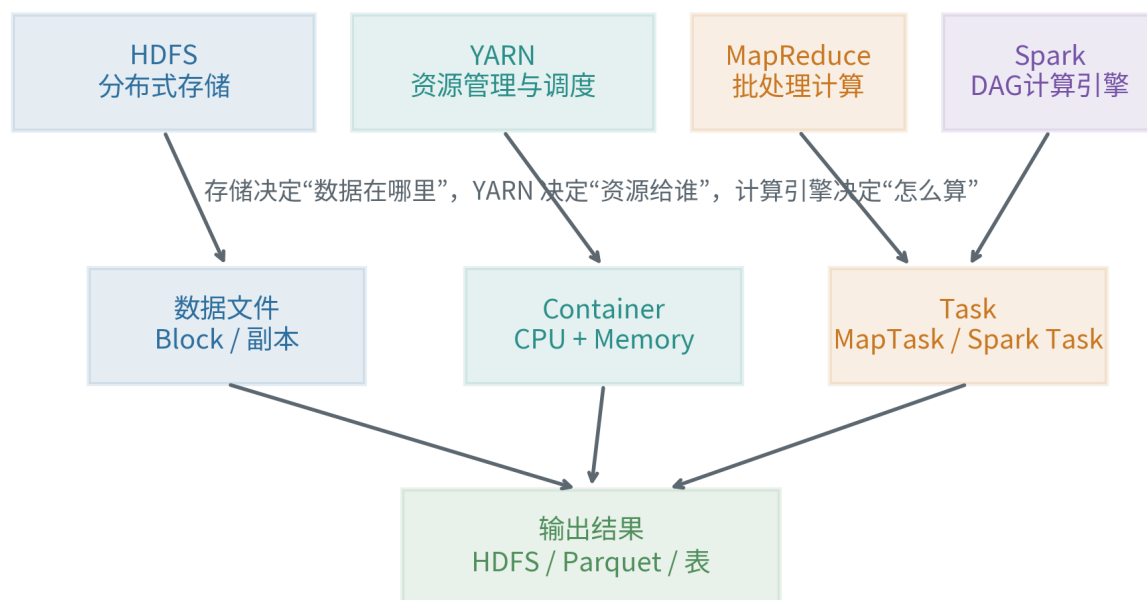
一、本讲到底要解决什么问题？

前19讲我们分别学了 HDFS、YARN、MapReduce、Spark、RDD、DataFrame、Catalyst、Shuffle、资源模型与调优。现在最容易出现的问题是：每个概念都懂一点，但它们在脑子里仍然是“碎片”。第20讲的任务就是把这些碎片压缩成一个统一系统。

你最终要形成的不是“会背定义”，而是下面四种能力：

能力	你应该能回答的问题
系统解释	一条数据为什么先放 HDFS？YARN 和 Spark 分别负责什么？
执行推演	一条 SQL 为什么会出现 Exchange？为什么会变成多个 Stage 和 Task？
故障定位	任务慢/失败时，是存储、网络、分区、资源、Join 还是内存问题？
技术迁移	以后学习 Flink、Ray、分布式数据库时，哪些思想可以直接复用？

Hadoop + Spark 统一知识地图



一句话先总括：HDFS 负责“存”，YARN 负责“分资源”，MapReduce/Spark 负责“算”；而真正昂贵的边界通常是 Shuffle。

二、HDFS：整个体系的“数据地基”

先从最底层开始。HDFS 解决的不是“怎么算”，而是“海量数据怎么可靠地放在很多机器上”。

2.1 你应该记住的四个关键词

关键词	底层含义	为什么重要
Block	大文件被切成较大的块分布存储	让文件可以跨机器存储，也方便并行读取
Replica	同一 Block 保存多个副本	机器损坏时仍能读到数据
NameNode	保存文件目录、Block 到 DataNode 的映射等元数据	告诉客户端“数据在哪里”
DataNode	真正保存 Block 数据	提供实际读写

2.2 为什么 HDFS 对计算模型有直接影响

MapReduce 的 InputSplit、Spark 的 Partition、数据本地性 (Data Locality) 看似属于计算层，实际上都和“数据在 HDFS 哪台机器上”密切相关。分布式系统的第一条工程直觉是：能移动计算，就尽量不要移动海量数据。

例如：1 TB 文件被分成多个 Block。如果一个 Task 能在存有对应 Block 的节点附近运行，就只需要搬“代码和少量元数据”；反过来如果总是把 1 TB 数据跨网络搬来搬去，网络会迅速成为瓶颈。

2.3 最简心智模型

HDFS = 分布式硬盘
NameNode = 目录和索引
DataNode = 真正装数据的机器
Block = 大文件切开的块
Replica = 为了故障而准备的冗余副本

三、YARN：整个体系的“资源调度层”

HDFS 解决“数据在哪里”，YARN 解决“谁能用多少 CPU 和内存”。YARN 并不知道你的业务是 WordCount、SQL 还是机器学习，它只负责资源。

组件	职责	类比
ResourceManager	全局资源管理、接收 Application	总调度中心
NodeManager	管理单台节点上的资源和 Container	每台机器的现场管理员
ApplicationMaster	为某个 Application 协调资源、跟踪执行	某个项目的项目经理
Container	一份被分配出来的 CPU + Memory 资源	可运行任务的资源盒子

3.1 一个关键纠错：Container 不是 Docker 容器

在 YARN 语境中，Container 首先是资源配额与执行环境。它表达“这个进程可以使用多少内存、多少 vCore，并在哪个节点启动”。不要把它和 Docker 的镜像/文件系统隔离概念混为一谈。

3.2 为什么 MapReduce 和 Spark 都可以跑在 YARN 上

因为 YARN 与具体计算框架解耦：MapReduce 可以申请 Container 跑 MapTask/ReduceTask；Spark 也可以申请 Container 启动 Executor。YARN 不替你做 Shuffle，也不替你执行 SQL。

因此可以把它们分成三层：

存储层：HDFS

资源层：YARN

计算层：MapReduce / Spark

四、一条数据从“存储”进入“计算”再回到“存储”

一条数据在分布式计算系统中的“生命周期”



把前面所有概念串起来，最有用的办法不是从 API 看，而是跟踪一条数据。

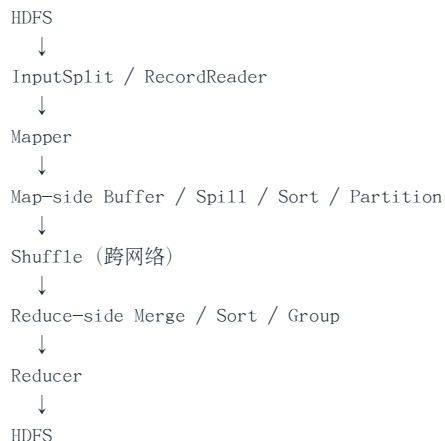
阶段	真正发生什么	常见成本
HDFS 读取	定位 Block，按 Split/Partition 读取数据	磁盘 I/O、远程读取
局部计算	每个 Task 对自己的数据做 map/filter/project	CPU、序列化
Shuffle	按 Key/分区规则把数据重新分发	网络、磁盘、序列化，通常最贵
聚合/Join	对“需要见面的数据”一起处理	内存、排序、Hash 表
结果输出	写 HDFS/Parquet 等	磁盘 I/O、小文件问题

真正值得形成直觉的是：局部计算往往便宜，数据跨机器重新组织往往昂贵。
所以后面无论学习 Spark、Flink
还是分布式数据库，“如何减少无谓的数据移动”都会反复出现。

五、MapReduce：固定但非常清晰的分布式批处理范式

MapReduce 的价值不在于“Map 和 Reduce 两个函数有多高级”，而在于它把大量分布式系统复杂性封装了：任务切分、调度、容错、Shuffle、排序、重试。

5.1 核心数据路径



5.2 MapReduce 的“科学问题”其实很朴素

它最适合的往往不是“单个计算极复杂”，而是“数据量极大、每条记录处理相对独立、最后需要按 Key 汇总”。例如日志统计、ETL、离线聚合、倒排索引、批量 Join。

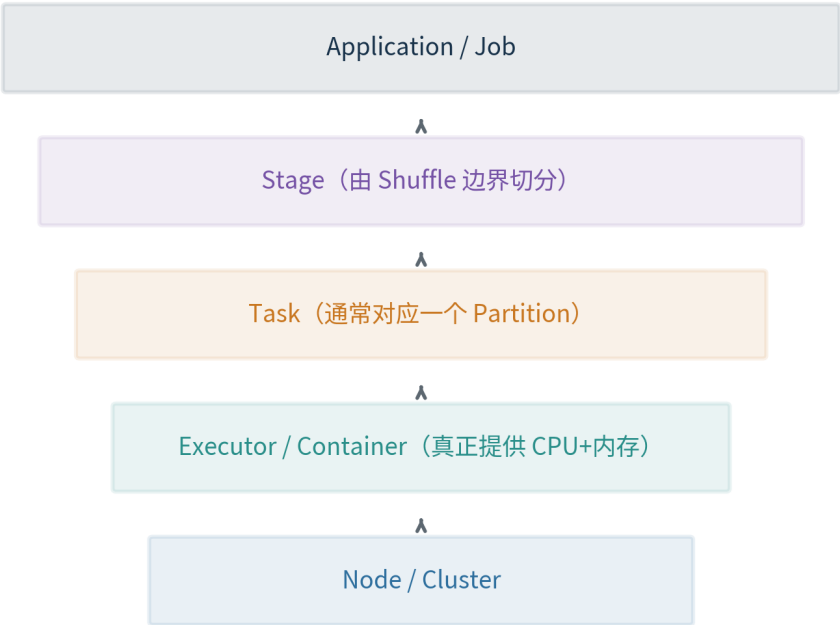
5.3 MapReduce 的限制

当任务需要多步计算、反复迭代或频繁复用中间结果时，固定的 Map→Shuffle→Reduce 模型会变得笨重，阶段之间经常需要落盘与重新读取。这正是 Spark 出现的重要背景之一。

六、Spark: 把固定阶段提升为 DAG

Spark 的关键进步不是一句“内存计算”，而是用 DAG 表达更一般的数据依赖，并在运行时把 DAG 切成 Stage，再把 Stage 切成 Task。

Spark 运行层级：从“逻辑任务”到“真实资源”



6.1 Transformation、Action 与 Lazy Evaluation

Transformation（如 filter、select、map）主要描述“想怎么算”；Action（如 count、collect、save）触发真正执行。因为中间不急着马上计算，Spark 可以先看到整条依赖链，再做优化和调度。

6.2 Narrow / Wide Dependency

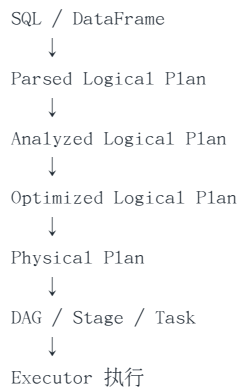
窄依赖通常意味着下游一个 Partition 只依赖少量上游 Partition，可在同一 Stage 流水执行；宽依赖意味着下游需要来自多个上游 Partition 的数据，通常产生 Shuffle，也通常形成 Stage 边界。

6.3 RDD、DataFrame 的差别

抽象	Spark 知道什么	优化空间
RDD	主要知道“这是一组分布式对象”和依赖关系	较低，很多计算语义对优化器不透明
DataFrame	知道列名、类型、表达式、Join、聚合等语义	更高，可以做列裁剪、谓词下推、Join 策略选择等

七、DataFrame + Catalyst: 从“代码”到“执行计划”

当你写 DataFrame 或 SQL 时, Spark 不会机械地逐行照抄执行。它会先把“你想做什么”转成逻辑计划, 再选择“怎么做”形成物理计划。



7.1 explain() 为什么重要

因为它让你第一次看到“Spark 真实打算怎么执行”。如果看到 Exchange, 通常要想到 Shuffle; 看到 BroadcastHashJoin, 说明一侧数据被广播; 看到 SortMergeJoin, 说明两侧通常需要按 Join Key 重分区和排序。

7.2 最值得记住的优化

优化	直觉
Predicate Pushdown	尽量早过滤, 少读无用数据
Column Pruning	只读取需要的列
Partial Aggregate	先在本地聚合, 再跨网络发送更少的数据
Broadcast Join	小表复制到各 Executor, 避免大表两侧一起 Shuffle
AQE	利用运行时统计在执行过程中调整计划

八、MapReduce 与 Spark: 不是两个世界, 而是同一问题的两种表达

MapReduce 与 Spark 的概念对照表

MapReduce	共同本质	Spark
InputSplit / MapTask	数据分片 + 数据并行	Partition / Task
Partitioner	按 Key 重分布	Partitioner / Exchange
Shuffle + Sort	网络重分布是昂贵边界	Shuffle / Exchange
Reducer	按 Key 聚合	reduceByKey / Aggregate
Job	一次批处理任务	Job
Map/Reduce 阶段	执行阶段	Stage / DAG

最重要的迁移能力, 是看到不同框架的名词以后, 能找到背后的共同机制。

例如 MapReduce 的 Partitioner 和 Spark 的 Shuffle

Partitioning, 本质都在回答: 某条中间数据应该发给哪个下游分区? ; MapReduce 的 ReduceTask 和 Spark 的下游 Task, 本质都要处理已经按分区规则重组过的数据。

再比如“数据倾斜”也不是 Spark 专属问题。只要一个 Key 的数据异常大, 就可能把某个 Reducer 或某个 Spark Task 拖成最后的长尾。

所以学习框架时不要只背名词。真正稳定的知识是: 数据分片、局部计算、重分布、聚合、资源分配、失败重算、状态与输出。

九、同一个销售额统计：MapReduce 和 Spark 分别怎么想

需求：10 亿条订单，计算每个商品的销售额总和。

9.1 MapReduce 思维

```
Mapper:
  读取一条订单
  输出 <product_id, amount>

Shuffle:
  相同 product_id 进入同一 Reducer

Reducer:
  sum(values)
  输出 <product_id, total_sales>
```

9.2 Spark DataFrame 思维

```
orders
  .filter('status = paid')
  .groupBy('product_id')
  .agg(sum('amount').alias('total_sales'))
```

代码形式完全不同，但底层依旧绕不开：读取数据 → 局部处理 → 按 product_id 重分区 → 聚合。Spark 的优势是它有更高层语义，Catalyst 可以帮助选择执行方式。

9.3 为什么理解 MapReduce 后学 Spark 会更扎实

因为你不会把 groupBy 当作“一个神奇 API”。你知道 groupBy 的背后通常需要让相同 Key 的数据真正“见面”，而在集群上让它们见面往往就意味着 Shuffle。

十、性能问题最终都可以压缩到几个物理成本

物理成本	典型症状	你应该看什么
磁盘 I/O	读写慢、spill 多、落盘多	输入格式、文件数、Spill、缓存策略
网络 I/O	Shuffle Read/Write 大	Exchange、groupBy、Join、Partition
CPU	Task 都忙但网络不高	UDF、复杂解析、序列化
内存	GC 高、Spill、OOM	Executor memory、缓存、Join 哈希表
调度开销	大量极短 Task、小文件	Partition 数、小文件、Task 数量
倾斜	大多数 Task 快，少数极慢	Task Duration、Records、Shuffle Read 分布

10.1 为什么“加机器”经常不是第一答案

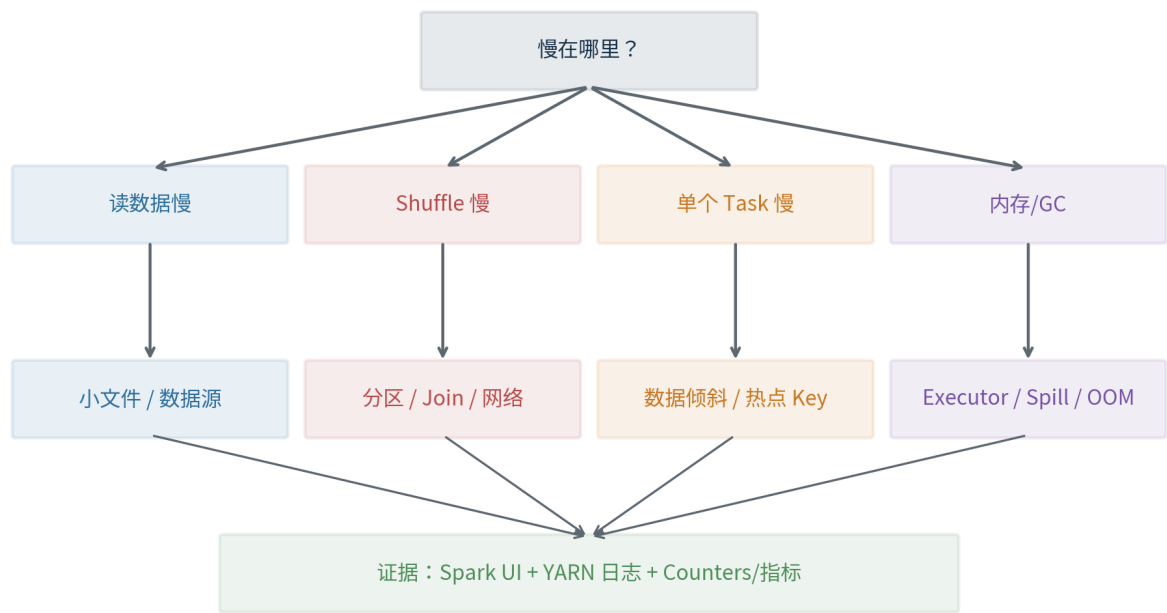
如果热点 Key 的所有数据必须进入同一个分区，那么即使增加 100 台机器，这个热点分区仍可能主要由一个 Task 处理。此时瓶颈不是“集群整体不够大”，而是“工作没有均匀分开”。

10.2 为什么先看 Spark UI

因为性能调优必须从证据出发。你应该先定位最慢 Stage，再看 Task 时长、Shuffle Read/Write、Input Size、Spill、GC Time、Records，最后才决定要不要改分区数、Broadcast、Cache、内存或代码。

十一、统一排错框架：不要先猜参数

统一排错思路：先定位系统层，再谈参数



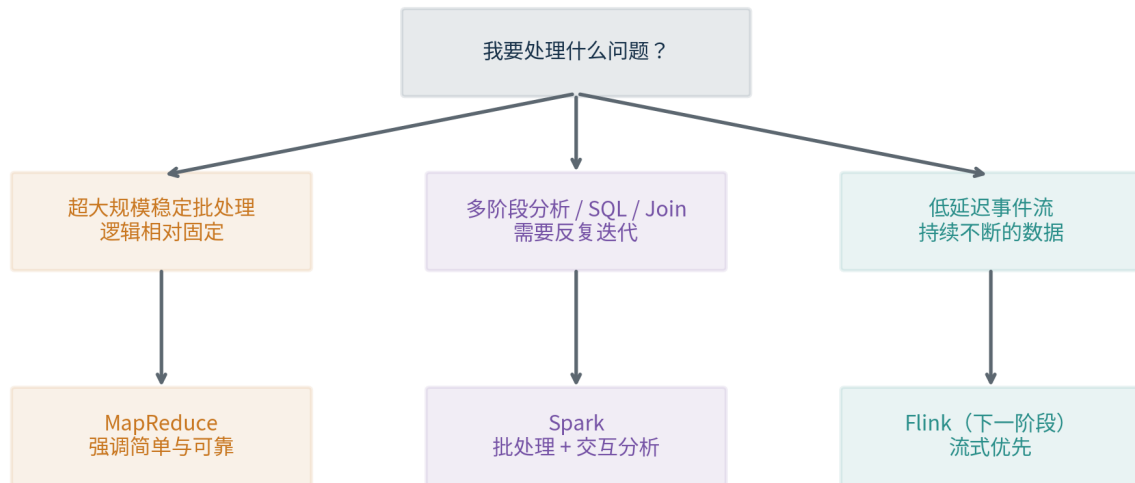
遇到任务慢或失败，可以固定按下面顺序：

层次	先问什么	典型证据
存储/输入层	文件存在吗？小文件多吗？Schema 对吗？	HDFS ls、Input Size、读取异常
执行计划层	是否出现大 Shuffle？Join 策略合理吗？	explain、Exchange、Join 节点
分区层	Task 是否均衡？是否热点 Key？	Task Duration、Records、Shuffle Read
资源层	Executor 是否频繁丢失？GC/OOM？	Executors 页面、YARN logs、exit code
业务正确性层	结果是否正确？	手算样例、Counter、聚合校验

非常重要：“Job SUCCESS” 只能说明框架认为任务成功结束，不等于业务计算一定正确。结果正确性仍需要手算样例、数据质量计数、结果校验等证据。

十二、什么时候用 MapReduce，什么时候用 Spark？

什么时候用 MapReduce、Spark，什么时候该想到 Flink？



在今天的新项目里，复杂批处理、SQL/ETL、交互分析通常更偏向 Spark；MapReduce 更多作为底层思想、历史系统或非常稳定的大规模批处理方案存在。

场景	更自然的选择	原因
超大批处理、逻辑简单、稳定优先	MapReduce 仍可胜任	模型简单、容错成熟、行为可预测
多步 ETL / Join / SQL 分析	Spark	DAG、DataFrame/Catalyst、表达能力高
迭代算法 / 重复复用数据	Spark	可缓存并复用中间结果
低延迟持续事件流	Flink 等流式系统	持续运行、事件时间、状态处理更自然

判断框架时不要问“哪个更先进”，而要问：我的数据是批还是流？延迟要求多少？计算是否多阶段？是否需要状态？是否有复杂 Join？

十三、你现在应该具备的“最低闭环能力”

学完这一阶段，不要求你记住所有参数，但至少能够独立完成下面这个闭环。

步骤	最低要求
1. 看数据	知道文件在哪里、格式是什么、规模多大、有没有脏数据
2. 建模	明确 Key / 分区 / Join / 聚合需要哪些数据见面
3. 写程序	能写基本 MapReduce 或 PySpark DataFrame 程序
4. 提交执行	知道 HDFS、YARN、spark-submit 的基本流程
5. 看执行计划	能从 explain() 识别 Exchange、Join、Aggregate
6. 看 UI	能定位慢 Stage 和异常 Task
7. 排错	能区分路径、Schema、Shuffle、倾斜、OOM 等类别
8. 验证结果	用小样例和计数验证，而不是只看 SUCCESS

如果这八步你能独立完成，那么你对 Hadoop/MapReduce/Spark 已经不是“听过”，而是形成了一个可工作的系统模型。

这也是后续学习 Flink 时最重要的基础：你已经知道分布式计算的物理世界里存在 Partition、网络重分布、Task、资源、故障和状态。

十四、综合推演练习

场景：你有 2 TB 电商订单 Parquet 数据，字段为 `user_id`、`product_id`、`city`、`amount`、`status`、`ts`。要求统计每个城市的已支付订单销售额，并找出每个城市销售额最高的 Top-10 商品。

请先自己回答下面问题：

问题	思考方向
Q1: HDFS 在这里负责什么？	保存 2 TB 数据及其 Block/副本
Q2: 哪些操作可以局部执行？	过滤 <code>status</code> 、列裁剪、 <code>amount</code> 清洗
Q3: 第一次 Shuffle 可能在哪里？	按 <code>city/product_id</code> 聚合时
Q4: 为什么 Top-10 可能需要第二阶段？	先局部聚合，再按 <code>city</code> 做排序/Top-N
Q5: 什么情况下会倾斜？	某个城市/商品极热
Q6: Spark UI 重点看什么？	最慢 Stage、Task 时长、Shuffle Read/Write、Records
Q7: 为什么 Parquet 比 CSV 更适合？	列裁剪、谓词下推、类型信息、压缩

参考执行思路

```
df = spark.read.parquet('/data/orders')

clean = (df
    .filter("status = 'paid'")
    .select('city', 'product_id', 'amount'))

product_sales = (clean
    .groupBy('city', 'product_id')
    .agg(sum('amount').alias('sales')))

# 再对每个 city 做窗口排序，取 Top-10
```

注意：真正重要的不是记这几行代码，而是能预测哪里会发生 Shuffle、哪里可能产生倾斜、哪里可以提前减少数据量。

十五、常见误区总复盘

误区	正确理解
“Spark 没有磁盘 I/O”	错误。Shuffle、Spill、输出、缓存级别等都可能涉及磁盘
“多加 Executor 一定更快”	错误。倾斜、单分区瓶颈、调度开销都可能让扩容无效
“groupBy 只是普通函数调用”	错误。在集群上通常意味着相同 Key 的数据必须重组
“HDFS = Hadoop 全部”	错误。HDFS 只是存储层
“YARN 会帮我优化 SQL”	错误。YARN 只管资源，Catalyst 才管 SQL/DataFrame 优化
“Job 成功就是结果正确”	错误。框架成功不等于业务正确
“Partition 越多越好”	错误。过多会增加 Task 调度、小文件和元数据开销
“RDD 比 DataFrame 更底层，所以更快”	通常不能这样推断。DataFrame 反而给优化器更多信息

判断是否真正理解的一道题

如果一个 Spark Job 有 500 个 Task，其中 498 个 20 秒内完成，2 个跑了 12 分钟，你首先应该怀疑什么？

优先怀疑数据倾斜或少数分区异常，而不是第一时间把所有 Executor 内存翻倍。

十六、把整个阶段压缩成 12 句话

1. HDFS 解决“海量数据如何跨机器可靠存储”。
2. YARN 解决“集群 CPU/内存资源如何分配给 Application”。
3. MapReduce 解决“海量数据怎样分散计算、按 Key 重组再聚合”。
4. 数据本地性是在网络昂贵的前提下，把计算尽量靠近数据。
5. Shuffle 是跨分区重新组织数据，是分布式计算最重要也最昂贵的边界之一。
6. Spark 用 DAG 表达比 Map→Reduce 更一般的计算流程。
7. Wide Dependency 通常意味着 Shuffle，也常意味着 Stage 边界。
8. 一个 Stage 内通常有多个 Task，每个 Task 处理一个 Partition。
9. DataFrame 让 Spark 看见列、类型和操作语义，因此可以自动优化。
10. Catalyst 把“我要算什么”变成“具体怎么执行”。
11. Spark 调优先看 UI 与执行计划，再改代码和参数。
12. 不论框架怎么变，分片、调度、数据移动、聚合、状态、容错这些底层问题不会消失。

如果以后你忘了很多 API，只要这 12 句话仍然在，重新学习 Spark/Flink 会非常快。

十七、下一阶段为什么进入 Flink?

MapReduce 和 Spark 主要让你建立了“批处理与 DAG”的完整基础。Flink

会把问题进一步推进到：如果数据不是一个已经放好的大文件，而是一条一条持续到来怎么办？

批处理视角	流处理视角
数据像“有限文件”	数据像“不会停止的事件流”
任务运行一段时间后结束	作业可能长期持续运行
重点是吞吐与批量计算	还要处理延迟、事件时间、乱序、状态
结果通常是一次性输出	结果可能持续更新

接下来学习 Flink 时，你会遇到新的关键词：

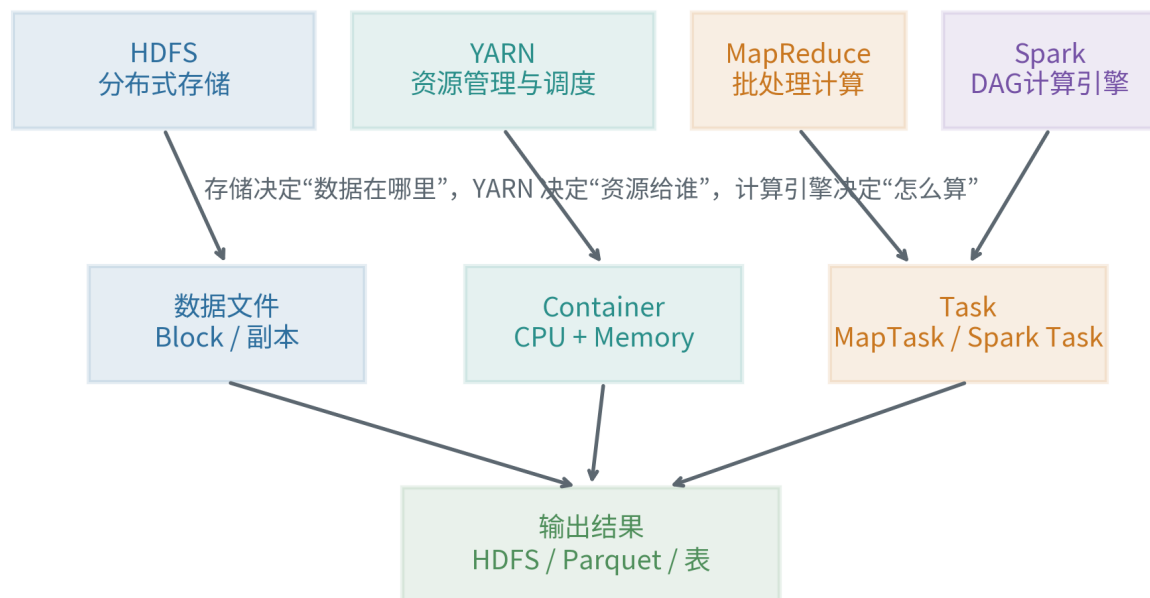
概念	先建立的直觉
Stream	持续到来的事件
Event Time	事件真正发生的时间，而不是机器收到它的时间
Watermark	系统对“事件时间进度”的估计
Window	把无限流切成可以计算的一段段有限集合
State	流式任务长期记住过去信息
Checkpoint	让有状态流计算能够从故障恢复
Exactly-once	故障恢复后仍尽量保证结果像“每条事件只处理一次”

你会发现，Flink 虽然术语变了，但

Partition、Task、Shuffle、资源、容错这些前面学过的底层概念仍然存在。

十八、阶段结课：把知识放回一张图

Hadoop + Spark 统一知识地图



最后再从系统角度复述一次：

数据先进入 HDFS
↓
YARN 为计算任务分配集群资源
↓
MapReduce 或 Spark 读取分区数据
↓
Task 做局部计算
↓
如果数据需要重新“见面” → Shuffle
↓
聚合 / Join / 排序 / Top-N
↓
结果重新写回 HDFS / Parquet / 表
↓
通过 UI、日志、计数与样例验证正确性和性能

到这里，Hadoop + MapReduce + Spark 阶段完成。后续进入 Flink 时，不会重新从“什么是分布式计算”开始，而是在这个模型上继续增加流式计算、事件时间、状态与 Checkpoint。

第20讲完